Railway Scheduling in Flatland with ASP

Marieke Warfia and Dominik Gerndt

University of Potsdam, Germany
marieke@warfia.eu, dominik.gerndt@gmail.com

Abstract. This report deals with the application of Answer Set Programming with Potassco Clingo [1] to the Vehicle Scheduling Problem (VSP) as presented in the Flatland Challenge [2]. First we introduce the Challenge and how Flatland and ASP can be used to find solutions. Afterwards we explore our three approaches Primitive, Primitive Prediction and Vertex that range from grid-based to graph-based solutions. Then we focus on the difficulties we encountered such as environment creation, spawning, time horizons and decision points. Lastly, we compare the performance of our approaches using hand-crafted environments and report about their limits and potential future work.

1 Introduction

This project's source code and wiki can be found on GitHub:
`https://github.com/warfiaUni/RailwayScheduling`

1.1 The Flatland Challenge

Flatland aims to be a framework for finding various different approaches to solutions of the VSP. More precisely, it abstracts railway scheduling into a grid-based representation and provides various ways to interact with the environment and simulate possible solution candidates. The goal is to have trains start at some initial position and let all reach their target destination within a given time horizon. An optimal solution would require the least amount of time-steps to reach this goal. However, as this is an NP-complete problem, the feasibility of finding optimal solutions decreases as the size of the environment increases. For this reason Flatland allows for customization of parts such as line generation, to specify specific routes and speeds, and observations, to increase or decrease the information each agent has available.

1.2 From Flatland to ASP

We expect basic knowledge of ASP and will therefore not explain the entirety of our implementation but focus on the general concepts and approach-defining rules. Since Flatland is built in python, we use the clingo python API in order to more directly interface between Flatland and ASP instances and encodings.

Modell	LibML	Laufzeit [s]	Durchsatz [pkts/s]	Mbits/sec
Snort ohne ML	-	45,77	203.707	1017,200
Tutorial-Modell	TFLite	57,79	161.324	805,400
Tutorial-Modell	FullTF	233,62	39.904	199,200
RDA (inkl. Preprocessing)	TFLite	48,48	192.300	960,200
RDA (inkl. Preprocessing)	FullTF	49,68	187.647	937,000
LSTM-Modell	TFLite	77,51	120.272	600,800
LSTM-Modell	FullTF	608,32	15.325	76,400

In this paper we will only focus on environments with global observation, which means the observation space is equal to the entirety of the environment. Moreover, we will not handle any dynamic behaviour that would require rescheduling, instead we define a single problem instance and apply one encoding from start to finish. For this reason, our problem instances are a direct ASP representation of very basic Flatland grid-worlds.

To describe a Flatland environment in ASP we read the necessary data directly from the environment object in python and transform it into a description using the literals *schedule*, *cell* and $diff$. The literal $schedule(ID, S, T, O, DP)$ represents an agent ID that starts at position S with orientation O, the agent has to reach the target at position T while only being able to depart after time step DP. A helper atom $diff(D, X, Y)$ describes the changes of X- and Y-coordinates when moving in direction D. While it was straight forward to implement the former two ASP representations, cells where more difficult, since all possible movements were encoded into a single 16-bit number. We will talk more about these difficulties in section 3.2. We represent a cell as $cell(P, O, D)$, where P and D are tuples. It describes a cell at position P on which a train with orientation O can move in any direction defined in D.

Listing 1.1: Example ASP instance of the environment shown in Fig.1

```
schedule(0,(1,0),(0,2),3,0).

cell((0,1),0,(1)).
cell((0,1),3,(2)).
cell((0,2),1,(3)).
cell((1,0),3,(1)).
cell((1,1),1,(0;1)).
cell((1,1),2,(3)).
cell((1,1),3,(3)).
cell((1,2),1,(3)).

diff(0, -1, 0).
diff(1, 0, 1).
```

```
14      diff(2, 1, 0).
15      diff(3, 0, -1).
```



Fig. 1: Simple Switch Map: A single agent starts at (1,0) with the target at (0,2) (coordinates are (Y,X), as defined in Flatland)

2 Approaches

In this section, we present our three approaches, or, to be more precise, our ASP encodings used to solve the problem instances in the format mentioned in Section 1.2. We will focus on the implementation and general ideas. In sections 3 and 4, analysis of difficulties, limits and performance will be discussed.

While the instance provides the same entry point for all encodings, due to the nature of ASP not all encodings necessarily produce a similar output. However, the environment simulation necessary for verifying the presence of an actual solution requires a specific input: a dictionary containing a Flatland-action for each individual agent to be simulated at a specific time step.

For this reason, we require all encodings to include $agent_action(ID, A, T)$ atoms in their stable models, where ID, A and T refer to the agent-identifier, Flatland-action and time step, respectively.

In our encodings we only use the four actions **left**(1), **forward**(2), **right**(3) and **halt**(4), we do not include **no-op**(0).

2.1 Primitive

Our first encoding, named Primitive due to its basic nature, aims to provide a straight forward solution. The idea is to let each train choose a direction, create transitions for this direction, exclude transitions that would result in collisions and continue to do so until the train reaches its destination. Each transition

is then transformed into a corresponding Flatland-action and included in the stable model.

More precisely, for each time step T, limited by some $limit(L)$, every agent ID at some position P with orientation O has to choose a direction D from $cell(P, O, D)$ or choose to stay at the current position. Given a choice of direction D, the rule shown in listing 1.2 creates a transition $trans(ID, P, P', O, T')$ from the old position P to the new position P' in the next time step T' while updating the agent's orientation O to the chosen direction D.

Listing 1.2: Agent transition from chosen direction

```
12 trans(ID,(X,Y),(X+A,Y+B),D,T+1) :-  trans(ID,_,(X,Y),_,T),
13                                      direction(ID,D,T),
14                                      diff(D,A,B).
```

If the new position P' corresponds to the agent's target, the agent is considered $done(ID, T)$.
The collision and *done* constraints (Listing 1.3) ensure no stable model contains transitions with vertex- or edge-collisions or agents that are not done by the time the limit is reached.

Listing 1.3: Encoding: Collision and finishing (done) constraints

```
19     %collision constraints:
20     :- trans(ID,_,B,_,T), trans(ID',_,B,_,T), ID!=ID'.
21     :- trans(ID,A,B,_,T), trans(ID',B,A,_,T), ID!=ID'.
22
23     % all schedules have to be done at some point
24     :- not done(ID,_), schedule(ID,_,_,_,_).
```

In order to identify the *agent_actions* corresponding to the generated transitions, we first count the valid directions in a cell given an orientation. This helps to differentiate between deliberate choices of **left** or **right** movement and a simple curve that results in an orientation change when moving forward.

Action Transformation Rules

- If the transition's start and end points are the same, the action was **halt**
- If the *count* is 1, the only action possible was **forward**.
- If the *count* is 2, but the orientation did not change, the only action possible was **forward**.
- If the action was not forward and the orientation changed by +1, the action was **right**
- If the action was not forward and the orientation changed by +3, the action was **left**

2.2 Primitive Prediction

While the primitive encoding was able to solve simple Flatland environments, we realized a significant performance issue. Given the rules of *primitive* every agent was able to choose an action at any point in time. In particular, halting was always an option, which resulted in agents frequently waiting when it was not necessary.

The first idea of *primitive prediction* is to remove choices except for when the agent actually has to pick a new direction. To achieve this, we used the *count* introduced in 2.1 to define **decision points** (Listing 1.4). A decision point is any cell where at least one agent-orientation has multiple valid transition directions, so any cell for which *count* is greater than 1 at least once.

Listing 1.4: Primitive Prediction: Definition of a decision point

```
4     decision_point(P) :- cell(P,O,D), count(P,O,C), C>1.
```

We then updated the *direction* choice rule to only apply to a **decision point**. If the agent is not at a decision point, then the agent directly transitions in the direction of travel (moves forward).

The second concept, the one responsible for *primitive prediction's* name, is to split transitions into **prediction** transitions and actual transitions. A prediction transition describes the movement the agent would make if it was an actual transition **without directly implying movement**. Instead, prediction transitions are used to predict the next time step. If two prediction transitions would result in a collision in the next time step, then all affected agents get the choice of either actually transitioning or halting.

Listing 1.5: Primitive Prediction: Transition choice in case of collision

```
45    1{
46        trans(ID,P,P',D,T);
47        trans(ID,P,P,O,T)
48    }1 :-   pred_trans(ID,P,P',D,T),
49            pred_coll(ID,_,T),
50            trans(ID,_,P,O,T-1).
```

If no collision would occur or the agent chose to transition, then the prediction transition is turned into an actual transition.

Listing 1.6: Primitive Prediction: Transition from prediction transition

```
42    trans(ID,P,P',D,T) :-   pred_trans(ID,P,P',D,T),
43                            not pred_coll(ID,_,T).
```

All agents are therefore only allowed to halt if and only if not halting would result in a collision, otherwise they always travel forward or choose a direction at a decision point.

2.3 Vertex

While *primitive prediction* was an improvement, it resulted in new problems. If two trains enter the same track segment from opposite directions, then this could potentially result in an unsolvable situation. Such an environment may require the agents to wait one segment earlier and let the other agent pass, but since *primitive prediction* forbids halting unless a collision is predicted for the next time step, halting at a previous segment is not possible.

To combat this, we decided on a **graph-based** approach. We used the previous idea of decision points and turned them into vertices of a graph. Furthermore, we turned start and target positions as well as the crossing section into vertices. The encoding then creates paths between these vertices and defines them as edges.

Our next ideas revolved around arrival and departure at vertices. At each vertex an agent may choose between departing from the vertex or remaining on the vertex. Similarly, each agent may choose to arrive at a vertex or halt one cell before a vertex. Therefore, all other cells imply forward movement. When an agent is located at a vertex, the **vertex** is considered **occupied**. To account for vertex collision it is thus sufficient, that no two agents can occupy the same vertex at the same time. We then decided that each **edge** is considered **blocked** if an agent is on any cell on the path defining the edge. Edge collision is therefore accounted for, if no two agents can block the same edge at the same time.

While this means that we now have a graph representation of the environment, our choice of edge collision still has some room for future work. In particular, since only two trains can occupy the same edge at any point in time, every environment that requires multiple trains to queue up in a single railway segment is not solvable by this encoding. An idea, that we did not get around to implement, was to actually build a queue on edges. More precisely, if two trains enter the same edge in opposing directions, then the edge collision is considered as previously and only one can enter. If however multiple trains enter the edge in the same direction, then a counter would specify the number of trains on the edge. Every train would then automatically drive forward until edge length *n-counter* and may only choose to continue if continuing would not result in a collision.

However, any approach allowing multiple trains on the same segment may result in long waiting times for multiple trains, when more realistic dynamic events like malfunctions are considered. This was our initial reason to not let any two trains on the same segment.

3 Difficulties

In this section we explain and analyse the difficulties we faced while developing the project. The goal is to build a knowledge base for students working on this project in upcoming semesters and help them getting started.

3.1 Map Generation

Before solving any environments, first these environments have to be created. In this section we focus on difficulties encountered when generating maps.

Specific Environments

Problem. Flatland currently provides a random map generator, while specific custom maps need to be generated with code. This is very verbose, hard to use and difficult to understand even for simple maps. However, we required specific maps for fast, repeatable and reliable testing of our encodings, especially considering edge-cases.

Solution. Our solution to this problem was a small custom-built environment editor[4] with an easy to use interface. This enabled us to quickly create the environments we desired, but it required extra resources to get started initially.

Starting Positions

Problem. In Flatland start and destination locations are tied to stations. These stations in turn are tied to cities, a form of container for multiple stations. Flatland randomly selects starting and destination points for agents from the pool of stations. Some of our maps however required specific start and destination points.

Solution. We opted for the simple solution of letting our map editor create a city wherever a station resides. This way we do not have to deal with cities except for a padding of one cell around every station. To assign our desired start and endpoints we created a custom line generator accepting a list of start and end points for each agent.

Oddities. Station orientations define the orientations of agents starting at them. More precisely, Flatland selects either the orientation or its opposite as the starting orientation of the agent. Additionally during simulation only endpoints are rendered as stations, which can at times be confusing.

Also, agents that have a dead-end as a starting position need to have its starting orientation facing into the dead-end. For example, if the opening of the dead-end was facing north, then the agent needs to face south into the dead-end, even if its intention is to drive north. This is something that is needed to be kept in mind when creating custom maps.

Another note is that, according to Flatland, dead-ends are not supposed to be used anymore and are not used in the random map generator anymore.

3.2 Translation between Flatland and ASP

In order to solve generated Flatland environments with ASP, all parameters relevant for solving have to be translated into an ASP instance. Likewise the solution generated by ASP needs to be translated back into a Flatland readable format.

Flatland to ASP.

Problem. It was difficult to understand how Flatland encodes its cells and grid and how the information can be extracted to form it into an ASP readable instance. In essence, Flatland encodes every cell in 16 bit with transitions that are possible on this cell. The following resource helps in understanding this issue: `https://flatland.aicrowd.com/environment/environment_information.html#get-full-transitions`.

Solution. We iterated over all cells and used bit-wise operations to extract the necessary information as described in Listing 1.7.

Listing 1.7: Primitive Prediction: Transition choice in case of collision

```
51 if not cell:
52     continue
53
54 for i in range(4):
55     facing_direction = (cell >> (12-4*i)) & 0xF
56
57     valid_directions = []
58
59     for j in range(4):
60         valid = (facing_direction >> (3 - j)) & 0xb1
61
62         if valid:
63             valid_directions.append(Direction(j))
64
65     # Create cell with Flatland style coordinates
66     # In Flatland Y goes to the right and X goes down
67     if len(valid_directions) > 0:
68         cells.append(cell_literal(
69             y, x, Direction(i), valid_directions))
```

First, we skip over all empty cells (lines 51-52), then we iterate over the 4 possible train orientations (line 54 onward). After extracting the values for the given orientation (line 55) we iterate over the possible directions (lines 59-63), where we obtain (line 60) and add (line 63) all valid directions. Lastly if the orientation in the current iteration has any valid directions (line 67), we create the cell literal (lines 68-69) and add (line 68) it to our description of cells.

ASP to Flatland.

Problem. Flatland has movements similar to the way a train moves, they are called actions (No-Op, Right, Left, Forward, Stop). This means a right turn without a switch is simply a "Forward"-action. The actions "Right" and "Left" are only needed with switches. In the beginning it was difficult to decide how to translate those actions into the encoding and back to Flatland in a good way.

Solution. We decided to require all encodings to include agent action literals, as described previously in (2), in their stable models.

3.3 Encodings

This section deals with specific difficulties regarding encodings and the solving process in general.

Horizon

Problem. Encodings have to use a limit to restrict the maximum amount of steps allowed for solving the map. Without a limit the solution space is endless. Is the limit too small, then the solution space may be empty. Since solving times depend on the size of the solution space, it is critical to find a limit as close to the optimum as possible. Setting a limit manually for maps poses no problem as long as we have only a set few simple maps. However, Flatland specializes in generating random maps, so automatically finding a horizon is preferred.

Flatland's Horizon. Flatland uses its own formula to calculate a horizon in the timetable generator. However, since the formula is based on shortest paths of the agents **disregarding collision** it is entirely possible to handcraft maps for which this horizon is too small. For instance, consider a map where two agents face each other on a straight line. At some point on the track is a simple switch. One of the trains has to take said switch in order to let the other train pass. If the straight line is considerably smaller than the detour required by the first train, then any horizon that just uses the shortest path with some multiplier may not be sufficient.

Solution. At this point, we were not able to find a good solution. We are using manually set limits for our custom maps and we are using the Flatland horizon for the random generated maps. A good way to circumvent the horizon problem would be to focus on a lower bound of the horizon and then use incremental solving.

Halting

Problem. The main issue in this project is to reduce the size of the solution space. If the solution space is smaller, then the performance improves and the encoding will be able to solve bigger and more complex maps. One of the main problems we were able to identify is halting. In our first encoding, "Primitive", the train considered stopping on every cell it encountered. This increases the solution space exponentially as map size and horizon increase. In this sense, it is an important question how to reduce unnecessary halting as much as possible, while still ensuring that most maps are solvable.

Solution. We had a few approaches to this issue that we implemented. For example, one improvement is to turn the rail network into a graph. Trains are not allowed to halt on straight paths. They are only allowed to halt in front of switches and can't drive onto an edge that is already occupied (for more information, s. [2.2] and [2.3]).

Future Work. A possible future approach could introduce costs/heuristics to every edge on the graph in order to improve decision making regarding waiting, compared to rerouting.

Departure and Horizon

Problem. Flatland calculates a range in which an agent can depart, then chooses a departure time randomly. Currently, this interferes with our manually set horizon. If the optimal path of a custom map is 20 steps, and an agent departs five steps later, it will need 25 steps.

Solution. We are currently manually setting all departure times to 0. A better approach would be to update the horizon accordingly or override departure times within Flatland.

3.4 Solution Verification

In this section we discuss the difficulties encountered during the solution verification process in Flatland. We verified our solutions by simply simulating the agent actions provided by the stable model.

Spawning

Problem. Agents in Flatland require a movement-based action to initiate a spawn and then need one step to spawn at its starting point. The encoding on the other hand assumes that the agent moved one step forward with this action.

Solution. To solve this problem, when simulating our environments, any non-spawned agent will use a Forward-Action[1] until it is spawned, afterwards the agent uses the actions according to the solution found by the encoding.

[1] s. rasch_simulator.simulate_actions()

Collision

Oddities. It is worth to keep in mind, that when a train drives into another train from behind, the train will automatically stop in the simulator. It is still possible that the train can continue. This information is interesting for debugging, because the issue can be an unnoticed collision. This results in the generated actions not fitting to the grid anymore, because they are simple action-commands for a single time step. On the other hand if two trains collide head-on the simulation will stop.

4 Benchmark and Limitations

In this section, we present the benchmarks of the encodings on a set of environments and we'll go over the limitations of each encoding.

Method (fig. 2 and 3). The environments are manually generated and each environment tests challenging or interesting cases. All environments are explained in detail in the GitHub-Wiki [5]. The statistics for the graphs are taken from `clingo.statistics`. If the encoding was not able to solve the environment in 60 seconds, the solving was terminated and received the flag "timeout". If the encoding solved the environment in time, but it was not able to find a solution, it received the flag "no actions". The graphs can be reproduced with the commands in Listing 1.8 The first command will benchmark and produce the statistics `all_stats.json` and the second command will visualise the JSON-file.

Listing 1.8: Reproducing fig. 2 and 3 with rasch.

```
rasch -b all -nr
rasch -v [path/to/all_stats.json]
```

Evaluation.

Primitive. The encoding Primitive was able to solve every environment except *20x20x1-snake*, due to a timeout. The snake-map is a very long track with no switches. It is particularly difficult for encodings, that have the choice to halt at every position. This shows the limitation of Primitive, that lies in generating too many choices where it is not necessary. The encoding's performance is scaling exponentially with the size of maps. It is also noticeable that it performs bad on *3x5x2-simple_order*.

Primitive-Prediction. This encoding performed the most unreliable. Environment *3x10x3-four_cycle* and *3x10x4-four_cycle* timed out while solving. *3x4x3-single_cycle*, *3x4x4-single_cycle*, and *3x5x2-simple_order* were not solvable by the

encoding. This is due to the limitation that Primitive-Prediction is only able to predict collisions one step into the future and will then allow halting to let an agent pass. If they need to halt earlier, the map is not solvable. The encoding has an advantage when the map is big, has only a few switches, and one agent, or two agents that need to go into the same direction. This is visible when comparing *3x10x1-four_cycle* and *3x10x2-four_cycle* with the encoding Primitive. As soon as two agents are present, the solving time of Prediction is worse than Primitive.

Vertex. This encoding performs significantly better in the parameters grounding and solving time than the other encodings 3. Even if it has on a few environments more choices and conflicts than Primitive, it is still overall faster (compare, *3x4x2-single_cycle* in fig. 2 and 3). It is also able to solve every map in the benchmarking. A limitation is that Vertex is not able to have more than one agent on one track moving in the same direction. As long as the trains can wait, the encoding is solvable.

Random Maps. The encodings were also tested on randomly generated maps, with a size of 24x24 only for solvability without further evaluation of statistics. All encodings were able to solve with two agents. Primitive and Primitive-Prediction performed slower than Vertex. Vertex was occasionally able to solve random maps with up to four agents.

5 Conclusion

In our report we explored Answer Set Programming as a viable option for solving the Vehicle Scheduling Problem within the context of Flatland. We presented our three approaches Primitive, Primitive Prediction and Vertex and analyzed the difficulties we encountered. After analyzing each encoding with the help of benchmarking on custom built environments, we think ASP shows promising results. However, despite the overall positive results, each encoding has clear limitations and room for improvement. Future work could focus on halting options for Primitive Prediction as well as allowing multiple agents on a single edge in Vertex. Other topics could include departure-time refinement, different halting strategies, heuristics and more advanced ASP techniques. Moreover, the Vehicle Rescheduling Problem (VRSP), due to its dynamic nature, could pose an exciting challenge in future projects.

6 Appendix

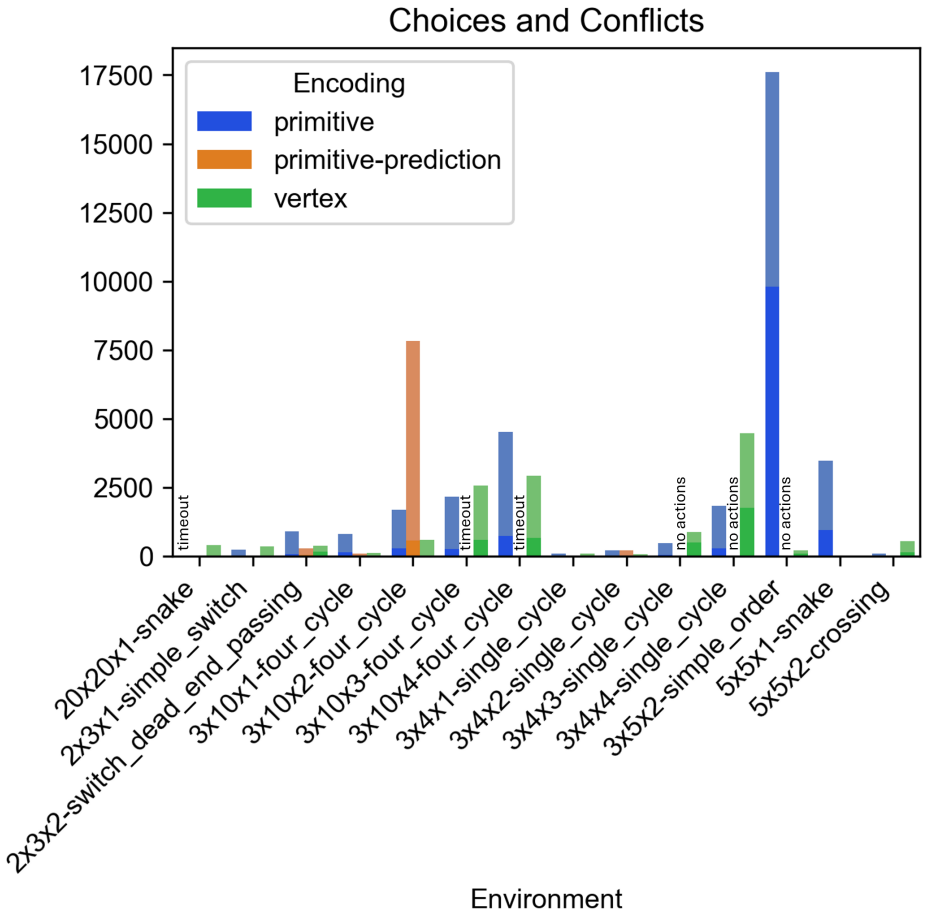


Fig. 2: Number of Choices and Conflicts for every Encoding and Environment

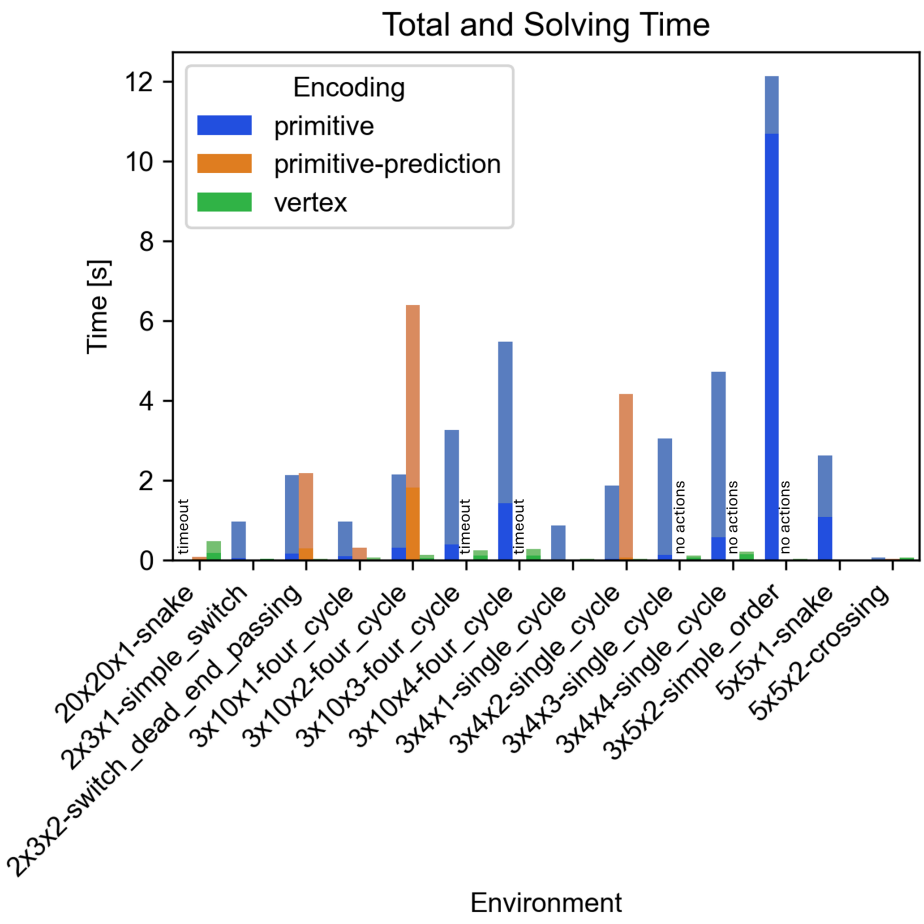


Fig. 3: Total and Solving Time for every Encoding and Environment

References

1. Potassco Clingo, `https://potassco.org/clingo/`
2. Flatland - Homepage, `https://flatland.aicrowd.com/intro.html`
3. RaSch Project - GitHub, `https://github.com/warfiaUni/RailwayScheduling`
4. Custom Map Generator - GitHub, `https://github.com/VictosVertex/flatland-asp-view`
5. Wiki Benchmark Environments - GitHub, `https://github.com/warfiaUni/RailwayScheduling/wiki/Test-Environments`